Milestone 3 Group 16

Website Github

Introduction:

The challenge of our selected project was creating models that were able to detect fraud in a dataset where only 1.1% of all transactions were fraudulent. In a dataset of nearly 1 million transactions, a model that ignored all fraud and labeled all transactions as non-fraudulent would perform with 98.9% accuracy. The shared goal for this assignment, the goal was to prioritize “recall” (catching fraud) over “accuracy.” For our Milestone 3 approach, our team implemented four distinct model combinations, leveraging various methods and combinations to detect fraud bank account applications within the Bank Account Fraud (BAF) dataset by Feedzai. Each member of the team implemented their own algorithm and technique. By doing this, we were able to compare supervised classification, along with unsupervised clustering and ensemble methods. Our approach includes Decision Trees, Support Vector Machines (SVM), Logistic Regression, K-Nearest Neighbor (KNN), K-Means Clustering, Apriori Association Rule Mining, and combination methods such as stacking and soft voting. This has allowed our team to evaluate which approaches are more effective at handling class imbalances.

Model Selection:

Prior to starting this project, we examined the root challenge of the problem detecting fraud in a highly unbalanced dataset. Instead of simply choosing and sticking to a single approach, we decided to choose models that have different strengths in an effort to compare approaches under such conditions and determine the best model for our data.

We chose to use **Decision Trees** because they give clear, rule-based splits that help us understand what features might contribute to fraud predictions. This method also does not require as much preprocessing and is simpler to use with mixed feature types.

Support Vector Machines (SVM) were chosen to separate data into groups methodically where there are a lot of features. In this case, our raw data set had over 30 features (both quantitate and categorical). The SVM method helps to catch fraudulent transactions in certain cases where a straight-line method might miss them.

K-Nearest Neighbors (KNN) was used as a distance-based approach. This model does not really “learn” one big model ahead of time. Instead, it compares the new tuple

to the stored data and decides based on similar cases which group it belongs to. This allowed us to test if local patterns were capable of identifying fraud.

Stacking and Ensemble Methods were used to combine the different strengths of individual models into a single model. Soft voting along with stacking was used to intelligently determine the weight of model outputs. Both of these models aimed to improve the overall robustness of the model.

K-Means Clustering was an unsupervised learning approach we decided to test out, not expecting it to provide very much information because of the nature of the data we are dealing with. It was used to see if fraud appears in identifiable segments.

Apriori Association Rule Mining was used because it helps identify easy to understand patterns in data instead of predicting. For example, we were curious to see if it was able to show if parameters like high credit risk and foreign requests appear together or not. This method helps provide very valuable insights into fraud analysis.

Performance Evaluation Metrics

To evaluate the performance of each of our models, we first needed to understand that a model with a 98.9% accuracy is very misleading. So, to evaluate our models correctly we relied on the following metrics:

Recall, the proportion of actual fraud cases correctly identified. This is arguably the most important metric because a missed fraud correlates directly to a financial loss. A high recall means that fewer fraud applications slip through the cracks.

Precision is the proportion or predicted fraud cases that were actually fraud. If a model had low precision, it means that there was an extremely high false alarm rate.

F-1 score considers both precision and recall. This score provides a single balanced measure. Very useful for comparing models that trade off these metrics.

Accuracy is the overall proportion of correct predictions. While we include this for completeness, accuracy is the least informative metric for this problem due to the extremely large amount of class imbalance our data set faces.

Confusion Matrices give us a breakdown of true positives, true negatives, false negatives, and false positives. It essentially shows where each model fails, so we can understand the tradeoffs of each approach.

Preprocessing Snapshots

The raw data consisted of 999999 rows and 32 features (both categorical and numerical). The fraud distribution in the dataset, as previously mentioned was 1.1029%, making for highly imbalanced classification of data. Two features were identified as having “high missingness” so those had to be managed in an appropriate manner. To handle this, the placeholder values present in the data (“-1”), were replaced with “NaN” values or the feature median values depending on the specific feature. Preprocessing was not conducted in the same manner for every single model, as certain models required different methods of data transformation. Other methods utilized in the preprocessing steps were encoding, scaling and balancing. Categorical variables were transformed via One-Hot Encoding, ultimately resulting in 51 features, as seen in Figure 2 below. Scaling was used in multiple models using “StandardScaler” to make sure that any of the distance-based models, such as SVM, KNN, etc., were not affected by any bias or outliers. Lastly, one set of models was conducted using a balancing method to see if that improved the model efficiency during the training process. This was done using Random Undersampling due to consistent timeouts and RAM limits when attempting to use Random Oversampling.

```
RAW DATA SNAPSHOT
Shape: (999999, 32)
Fraud distribution:
fraud_bool
0    0.988971
1    0.011029
Name: proportion, dtype: Float64
Missing values per column:
prev_address_months_count    712919
bank_months_count            253635
device_distinct_emails_8w       359
```

Figure 1: Preprocessed Data Snapshot

```
PROCESSED DATA SNAPSHOT (after imputation, scaling, encoding)
Shape: (799999, 51)

Feature count: 51
Missing values: 0
```

Figure 2: Post-Processed Data Snapshot

Model 1 (Samuel Meaux): Class Imbalance Handling using Decision Tree, Support Vector Machine, Ensemble and Stacking

For this analysis, nine modeling architectures were tested but ultimately only 5 were kept: Decision Tree, Support Vector Machine (SVM), Logistic Regression, Ensemble (Soft Voting), and Stacking. These models were selected to determining the ultimate method for determining fraudulent account creation methods in an imbalanced dataset utilizing an artificially balanced training set.

Implementation and Preprocessing:

- **Missing Data Signals:** At the beginning of the preprocessing pipeline, placeholder values of "-1" were replaced with NaN and "missingness" indicator flags were created for the features "prev_address_months_count" and "bank_months_count." This method took the data and treated all the missing values as a feature that is behavioral in nature rather than just plain noise.
- **Feature Selection:** Using the LassoCV function, the data was streamlined to the most significant features. This, however, only tailored down the total number of features from 49 to 47, which was surprising. Even though it wasn't tailored much, this method was able to ensure that all future computations and all future and model implementations were conducted efficiently.
- **Class Imbalance Handling:** Only 1.1% of the entire dataset were fraudulent transactions. To manage this, Random Undersampling (RUS) was utilized to build a balanced training set (11,029 samples per class). This was critical in preventing the models from ignoring the examples of fraud. Initially, Oversampling was attempted but caused extremely lengthy model run times and was subsequently removed from our handling methods.

Model Assumptions and Adjustments:

- **Decision Tree:** A max depth of 12 was found to provide the highest accuracy and provided a clear and rule-based baseline, while at the same time, was capable of preventing overfitting.
- **SVM:** It was assumed that the "separability" was none linear for this SVM model. Due to the complexity associated with this large amount of features and such limited examples of fraud, it was determined that a standard linear hyperplane would not be adequate for this model. In order to distinguish classes, a radial basis function (RBF) kernel was used in combination with a balanced class weight standard to capture any non-linear fraud patterns. The features were

scaled using the “StandardScaler” function to ensure the SVM was capable of accurately calculating all of the distance boundaries present in the data.

- **Logistic Regression (Linear Model):** It was assumed that the independent variables and the “log-odds” of the target class had a linear relationshiop for this model. In order to meet standalone linear model requirements, LR was done using a StandardScaler with a max iterations of 1,000 in order to ensure a convergence was observed.
- **Aggregations:** A Soft Voting Ensemble (voting based on probabilities) was implemented based on the assumption of model variety. By choosing to utilize three different model types (Decision Tree, SVM, and LR models) this aggregation method effectively determined from the accuracy/well-calibrated probabilities which result was the ultimate correct choice. Additionally, a Stacking Classifier was developed using a Logistic Regression component as a “meta-learner” to smartly collect the outputs of the primary models and weigh them accordingly.

Balanced Testing - Confusion Matrix (Stacking Model):

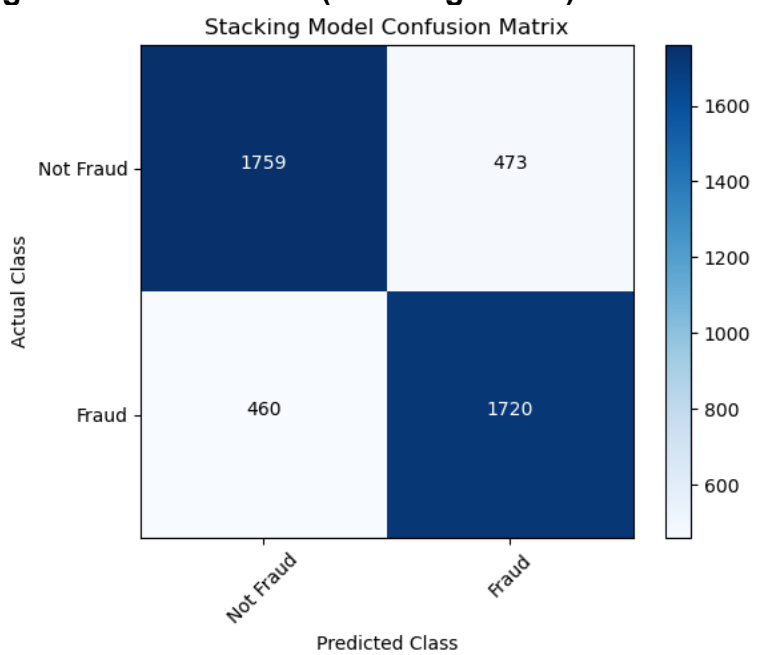

Figure 3: Stacking Confusion Matrix (Balanced Test Set)

Model Comparison of Performance (Balanced Test Set):

Model	Accuracy	Precision (Fraud)	Recall (Fraud)	F1-Score (Fraud)
Decision Tree	73%	71%	75%	73%
SVM (RBF)	79%	79%	79%	79%
Logistic Regression	79%	78%	79%	79%
Ensemble (Voting)	79%	78%	79%	78%
Stacking (Best)	**79%**	**78%**	**79%**	**79%**

Table 1: Model Comparison of Performance (Balanced Test Set)

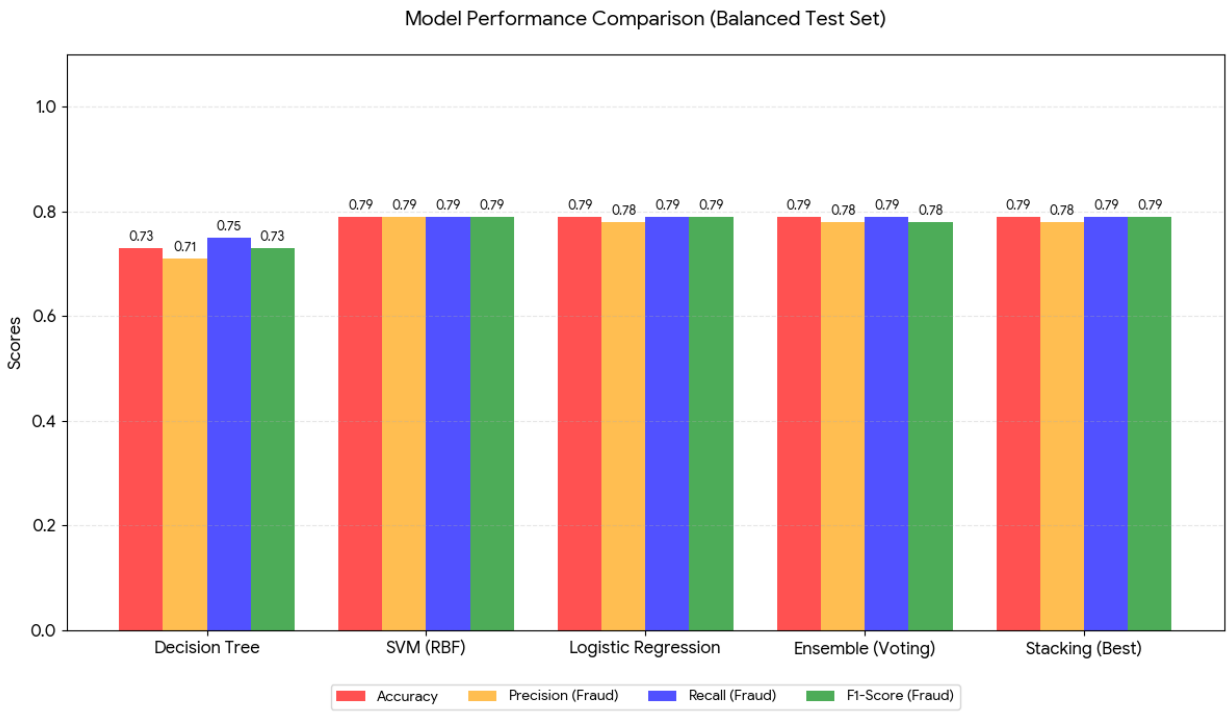

Figure 4: Model Comparison of Performance (Balanced Test Set)

Unbalanced/Full Testing - Confusion Matrix (Logistic Regression Model):

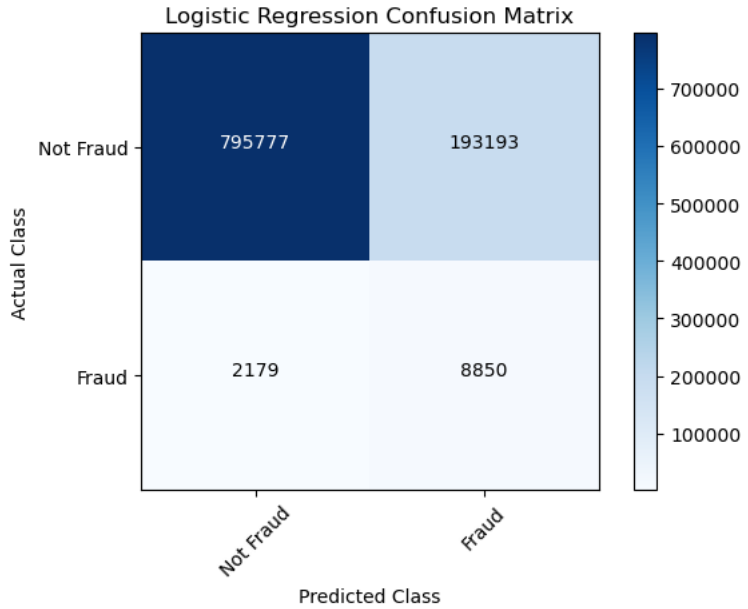

Figure 5: Logistic Regression Confusion Matrix (Full/Imbalanced Test Set)

Model Comparison of Performance (Balanced-Tested Models with Imbalanced/Full Data Set):

Model	Accuracy	Precision (Fraud)	Recall (Fraud)	F1-Score (Fraud)
Decision Tree	**72.8%**	**3%**	**83%**	**6%**
SVM (RBF)	55.7%	2%	95%	5%
Logistic Regression (Best)	80.46%	4%	80%	8%
Ensemble (Voting)	59.02%	2%	94%	5%
Stacking	55.52%	2%	95%	4%

Table 2: Model Comparison of Performance (Balanced-Tested Models with Imbalanced/Full Data Set)

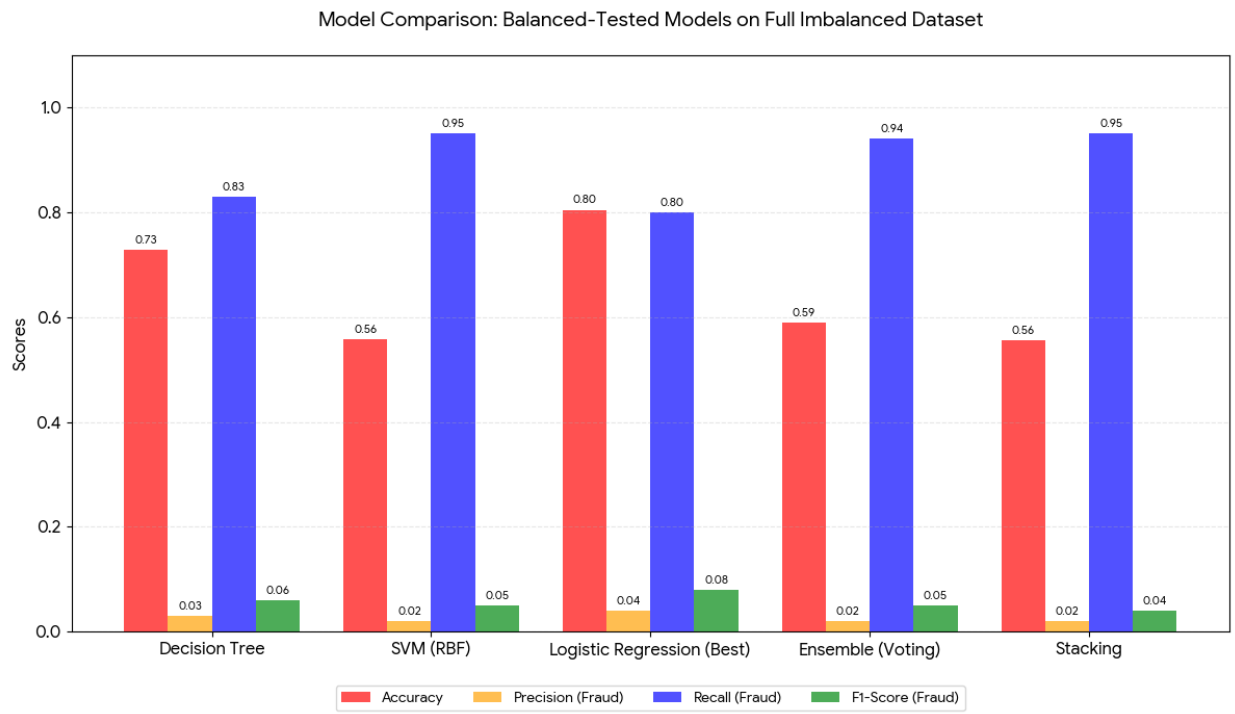

Figure 6: Model Comparison (Balanced-Tested Models on Full Imbalanced Dataset)

Issues and Improvements: The first issued occurred while training SVMs on a large dataset. When attempting to train on a Random Oversampled data set, the model did not finish and was ultimately not usable. Because of this, in the rest of the models, RUS was implemented. This ended up providing a high recall of 79% (Stacking) while maintaining manageable training times. Similar results were found in the full testing (high recall) yet there were low F1-Scores in the full testing. Future models will seek to improve F1-Scores and reduce the amount of “false alarms” that were so heavily prevalent in these models.

Class Imbalance Handling Conclusion: Through the combination of feature optimization, ensemble methods and stacking methods of Logistic Regression, SVM models and decision tree models, there was a significant improvement in fraud identification in the balanced test data. The final “Stacking Model” in the balanced testing set was able to effectively capture almost 80% of the fraudulent activity. This provided a reliable and balanced tool for detection in this dataset. However, in the unbalanced/full testing set, it was found that the Logistic Regression model performed subjectively better than the other three models with the highest accuracy (over 80%) and highest F1-Score (8%). That said, there was still a large portion of non-fraudulent bank account initiations that were flagged as fraudulent; something that needs to be addressed in further optimization of these models.

Model 2 (Kayo Abdi): Fraud Detection using Decision Tree, K-Means Clustering, and Apriori

Implementation and Preprocessing: For this set of models, the dataset was preprocessed by replacing values -1 with NaN to represent data that was missing. For numerical features, median imputation was applied. For key attributes such as bank account durations and previous addresses, however, missing values were treated as meaningful indications rather than just noise. One-hot encoding was used on categorical variables, which gave a total of 47 features and the final dataset contained just 1,000,000 observations.

Model Selection: Three different modeling methods were used including, Decision Tree classification, K-means clustering, and Apriori association rule mining. Each model was selected to capture different aspects of fraud detection. Thus including behavioral grouping, predictions, and pattern discovery.

- **Decision Tree:** The decision tree model was chosen due to its easy interpretability and suitability for classification. This model assumes that the data can be partitioned into different decision regions based on feature thresholds. To reduce overfitting, the maximum tree depth was limited to 5. Also, due to the severe class imbalance of about 1.1% fraud, class weighting was applied to highlight the minority class (aka fraud). The model achieved a recall of around 74% for fraud cases, thus representing effective detection of fraudulent activity. But precision was low by around 3%, reflecting a high number of false positives. The confusion matrix shows that most non-fraud cases are correctly classified, but a large number are incorrectly predicted as fraud, while only a smaller portion of fraud cases are missed. But after some research it was discovered, that this trade-off is expected in fraud detection scenarios where minimizing missed fraud cases is prioritized. **This suggests that the model is more focused on capturing as many fraud cases as possible, even if it means incorrectly flagging legitimate users, which aligns with the goal of reducing financial risk.**

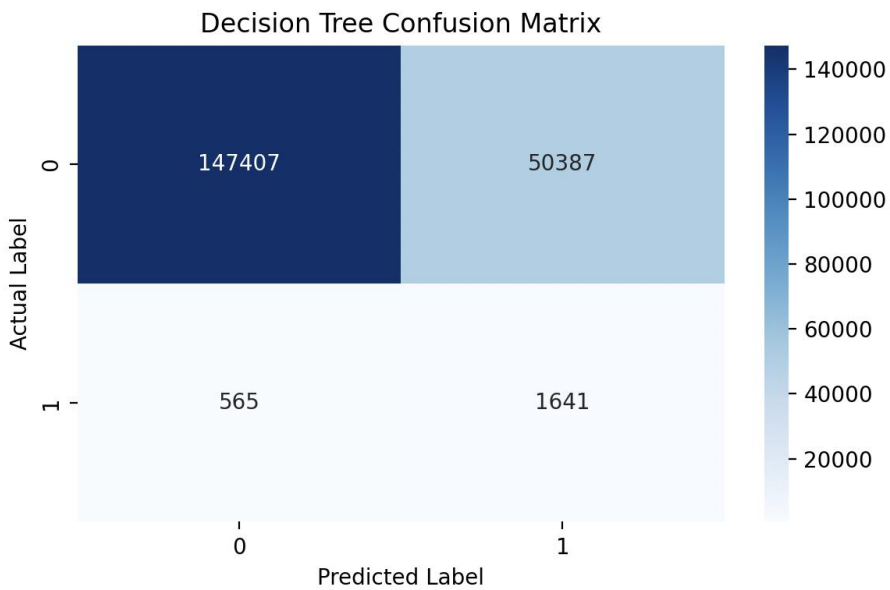

Figure 7: Decision Tree Confusion Matrix

```
Classification Report:
              precision    recall  f1-score   support

           0     0.9962    0.7453    0.8526    197794
           1     0.0315    0.7439    0.0605      2206

    accuracy                         0.7452    200000
   macro avg     0.5139    0.7446    0.4566    200000
weighted avg     0.9855    0.7452    0.8439    200000
```

Figure 8: Decision Tree Classification Report

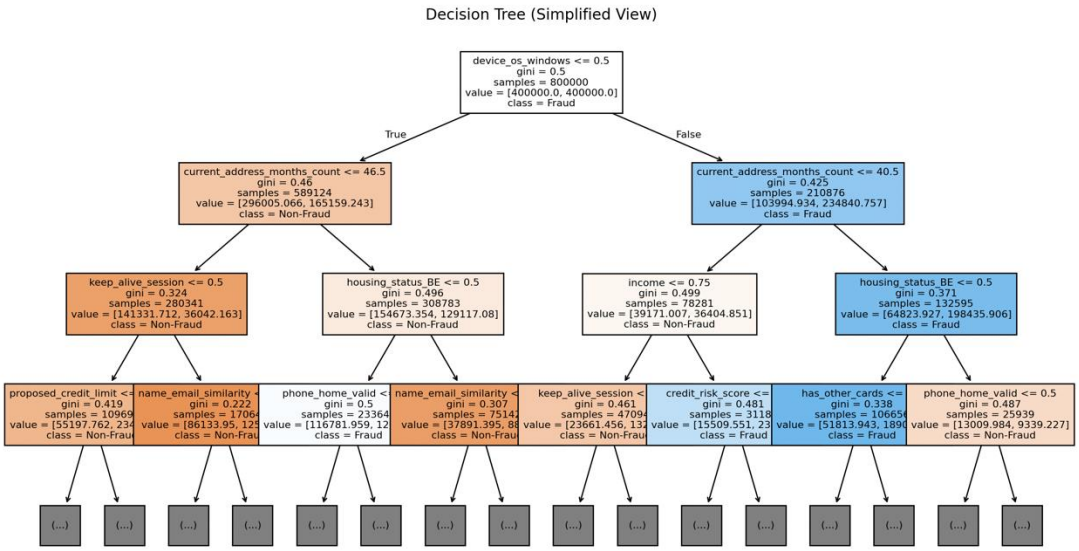

Figure 8: Decision Tree Visualization

- **K-Means Clustering:** K-means clustering was applied to find natural groups in customer behavior. The model assumed that clusters are formed based on distance in feature space and that groups are approximately spherical. Prior to clustering, numerical features were standardized, and a subset of the data was used to improve computational effectiveness. The results showed distinct clusters with varying fraud rates. One cluster showed a much higher fraud rate about 2.24% compared to others that were around 0.63%–0.98%, thus suggesting that fraud is grouped in specific behavioral segments. The silhouette score of 0.076 indicates that while clusters are present, there is notable overlap between them and the separation is relatively weak. The Davies-Bouldin index (3.02) also supports this, indicating that the clusters are not very compact and show some overlap with each other

Figure 9: Fraud Rates by Cluster

- **Apriori:** Apriori association rule mining was used to show frequently co-occurring feature association. This method assumes that frequent itemsets indicate meaningful relationships, **and relies on the anti-monotone property where infrequent itemsets are pruned from further expansion**. Due to the computational constraints and dataset size, the model was applied to a sampled and discretized subset of the data. Numerical features were transformed into categorical bins to support transaction-based analysis, **since Apriori requires categorical input and treats each record as an independent transaction**. But, due to the rarity of fraud cases, many rules involving fraud did not satisfy the minimum threshold.

Conclusion: Overall, the Decision Tree model was the best at predicting fraud and K-Means helped us understand the structure of the data. Apriori analysis demonstrated the importance of feature interactions. The visualizations show that there are clear patterns in how fraud appears within the data. The confusion matrix indicates that the model is able to capture a decent amount of fraud cases, but it also captures a large number of non-fraud cases as fraud. This shows a tradeoff between catching fraud and making incorrect predictions. The Decision Tree visualization shows that splits are based on both behavioral and financial features, which suggests fraud isn't driven by

just one factor. This suggests that fraud is influenced by more than just one type of variable. The clustering results show this in the PCA plot, where the data breaks into three groups, even though there's still some overlap. When we look at the rate of fraud in each group, we see that one group commits more fraud than the others. Hence, it is not random. It seems that fraud is more common in certain parts of the data.

Model 3 (David Savić): Fraud detection using Decision Tree, Naïve Bayes, Apriori, and Logistic Regression

Model Selection and Preprocessing: Four models were implemented on the Feedzai Bank Account Fraud dataset (999,999 rows, 1.10% fraud rate). Similar to the models above, the same preprocessing pipeline was used throughout: placeholder "-1" values replaced with "NaN", median imputation on numeric columns, most-frequent imputation and OneHotEncoding on categorical columns, and StandardScaler on numeric features - yielding 51 features after encoding. An 80/20 stratified split was applied across all models.

Decision Tree: The Decision Tree was the first choice because of the simplicity of output interpretation and explanation. Every prediction can be traced back to a specific split on a specific feature, which is extremely important when trying to justify a fraud flag. Decision Trees also handle the mix of numeric and encoded categorical features without much extra setup. Additionally, "class_weight" lets you penalize the model directly for missing fraud cases.

Decision Tree Model Assumptions and Tuning: Trees split data into rectangular regions using single-feature thresholds. The big risk here is overfitting across 51 features, especially the sparse one-hot columns. After using "GridSearchCV" with 3-fold CV and F1 scoring across 96 combinations of "max_depth," "min_samples_leaf," "criterion," and "class_weight," the best outcome was "max_depth=5," "min_samples_leaf=10," "entropy," and "class_weight={0:1, 1:10}." The custom 10x fraud weight beat the "balanced" option. The balanced setting auto-calculated to roughly 90x on this dataset, which pushed recall up but created a large amount of false positives that reduced the final F1-Score.

	Accuracy	Recall	Precision	F1	ROC-AUC
Baseline	0.9796	0.0784	0.0777	0.0781	--

Tuned	0.9652	0.2625	0.0979	0.1426	0.8051

Table 3: Decision Tree Classification Report

The baseline model predicted “not fraud” for almost everything and hit 97.96% accuracy anyway — which is exactly why accuracy is the wrong metric here. After tuning, recall went to 26.25% and F1 to 0.1426. Out of 2,206 fraud cases in the test set, the model caught 579, missed 1,627, and threw 5,333 false positives.

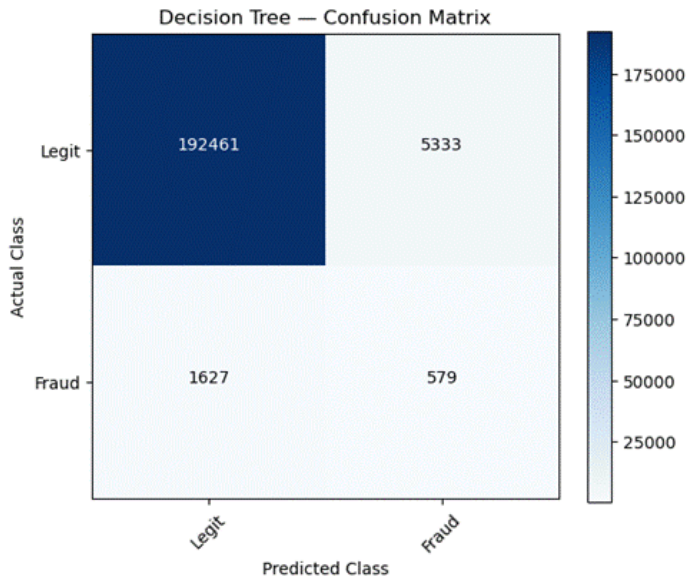

Figure 10: Decision Tree confusion matrix (tuned model)

Housing status (housing_status_BA at 0.43) and device OS (device_os_windows at 0.21) dominated the importance scores, which was a bit unexpected. Behavioral features like name_email_similarity and velocity showed up further down the list. The tree may be picking up on patterns in the encoded categories rather than purely behavioral signals, which would be worth investigating further.

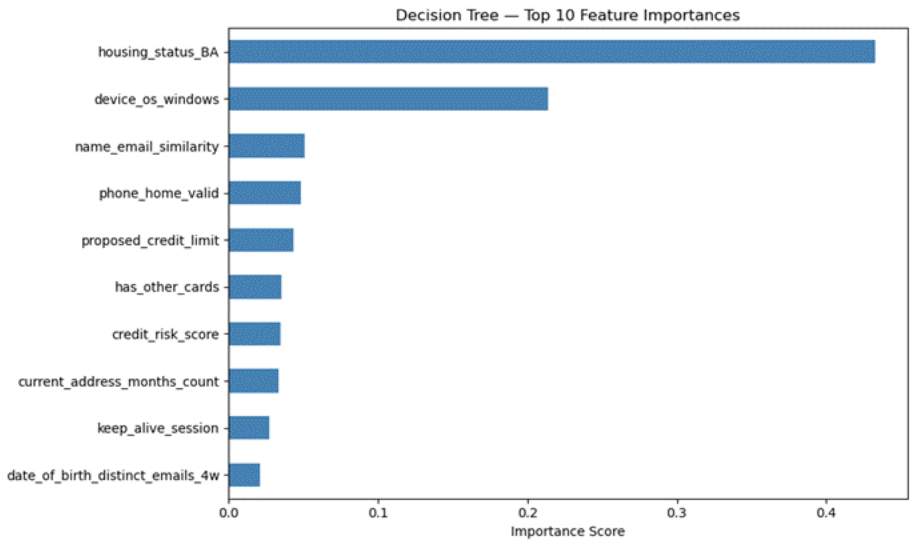

Figure 11: Top 10 feature importances

Naïve Bayes

Naïve Bayes was included mainly because it gives a fraud probability score rather than just a binary label. That is useful in practice — a fraud team can lower the threshold to catch more fraud at the cost of more false alarms, or raise the threshold to reduce noise. Retraining is not necessary, just shift the cutoff.

Model assumptions and tuning: GaussianNB assumes features are independent given the class and normally distributed within each class. The independence assumption is definitely violated since velocity_6h and velocity_24h are correlated, but the model still works well enough as a probabilistic baseline. Since GaussianNB does not support “class_weight,” the imbalance was handled by adjusting the decision threshold after prediction.

	Accuracy	Recall	Precision	F1	ROC-AUC
Baseline	0.9340	0.3409	0.0602	0.1023	0.8236
Tuned (var_smoothing=1e-7)	0.9808	0.1927	0.1708	0.1810	0.8309

Table 4: Naïve Bayes Classification Report

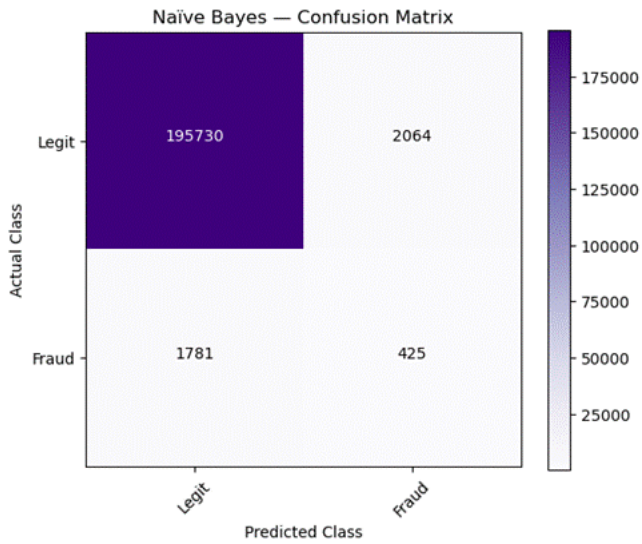

Figure 12: Naïve Bayes confusion matrix

This tuned model got the best F1 of the three classifiers at 0.1810, with a ROC-AUC of 0.8309. 425 out of 2,206 fraud cases were caught with 0.5 threshold as default. Dropping the threshold to 0.10 raises that to 1,104 caught — but also adds around 12,000 more false alarms. Whether that tradeoff makes sense depends on how the bank weighs the cost of a missed fraud versus an incorrectly flagged application.

Threshold	Recall	Precision	F1	Fraud Caught	False Alarms
0.50	0.1927	0.1708	0.1810	425	2,064
0.30	0.2883	0.1183	0.1677	636	4,742
0.10	0.5005	0.0734	0.1280	1,104	13,941
0.05	0.6097	0.0560	0.1027	1,345	22,652

Table 5: Tuning Results Matrix

Apriori: Apriori is not a predictive model — it does not output a fraud label or score. The point of including it is to find which feature combinations tend to appear together in fraudulent applications and express them as readable rules. That is actually more useful for analysts than a black-box classifier in some cases.

Setup: All continuous features had to be discretized first since Apriori works on categorical items. Income and “credit_risk_score” got binned into three quantile buckets (low/mid/high), which is the same approach used in the “Class Imbalance Handling” model section above. Velocity features were rank-binned the same way. Binary flags like “phone_home_valid” and “foreign_request” were kept as-is. The fraud class (11,029 rows) was mined separately rather than the full dataset because at full-dataset scale, the 1.10% fraud rate makes the support thresholds too restrictive to surface any fraud-related rules. A 50,000-row legit sample was mined in parallel as a comparison.

Top Rules (abbreviated)	Support	Confidence	Lift
velocity6h_high, no foreign req -> velocity24h_high	0.1507	0.5379	1.936
velocity24h_high -> velocity6h_high	0.1579	0.5375	1.919
no home phone, vel24h_low -> mobile phone present	0.1515	0.5762	1.708
no other cards, vel24h_low -> no foreign req	0.1656	0.5149	1.620

Table 6: Top Apriori Rules

687 frequent itemsets and 105 rules came out of the fraud class at the set thresholds (support >= 0.15, confidence >= 0.50, lift >= 1.20). The high-velocity co-occurrence rules dominated the top results. There was also a second cluster around “no home phone,” “mobile present,” and “low velocity,” which reads like a new account making small, cautious transactions before committing to larger fraud. No features were exclusively frequent in fraud and absent from legit transactions, which reflects the general behavioral overlap between the two classes and explains why this problem is difficult.

Logistic Regression: Logistic Regression was included as the linear baseline and also because it is used as the meta-learner in our stacking ensemble. It outputs well-

calibrated probabilities across the full range, which is what makes it a good choice for combining base model outputs. L1 regularization can also zero out irrelevant features automatically across the 51 columns.

Model assumptions and tuning: The model assumes log-odds are linearly related to the features. The velocity features are correlated but regularization handles that. Features must be scaled first — StandardScaler was applied to all numeric columns, which the snippet below shows. GridSearchCV ran 5-fold CV over C in [0.001, 0.01, 0.1, 1, 10] and L1 vs L2 penalty. C=0.01, L2 was the winner.

Feature	Before StandardScaler	After StandardScaler
income	0.1000	-1.5942
velocity_6h	8,892.008	1.0717
proposed_credit_limit	200.00	-1.12 (approx.)

Table 7: Tuned LR Outputs

	Accuracy	Recall	Precision	F1	ROC-AUC
Baseline (C=1, L2)	0.7909	0.7239	0.0373	0.0710	0.8175
Tuned (C=0.01, L2)	0.8065	0.7874	0.0435	0.0824	0.8763

Table 8: LR Classification Report

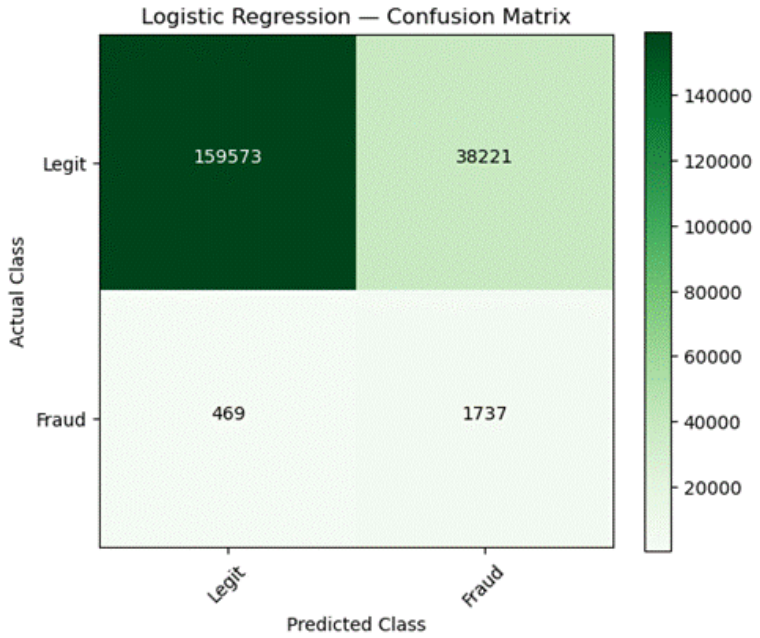

Figure 13: Logistic Regression Confusion Matrix (tuned, C=0.01)

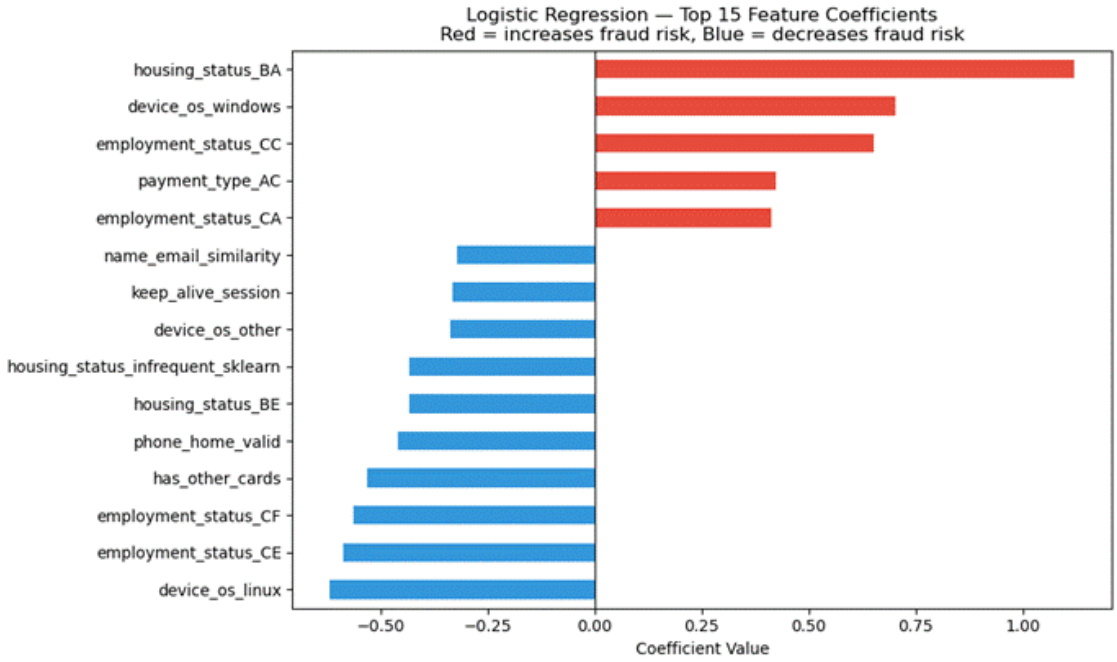

Figure 14: Top 15 Feature Coefficients (red = increases fraud risk, blue = decreases)

The tuned model caught 1,737 of 2,206 fraud cases (78.7% recall) with 38,221 false positives. The high false positive count is a direct result of “class_weight=balanced” on a 90:1 dataset — the model is heavily penalized for missing fraud, so it over-flags legitimate applications. The coefficient chart shows “housing_status_BA” and “device_os_windows” as the strongest positive predictors of fraud, consistent with what

the Decision Tree found. On the negative side, “employment_status_CE” and “device_os_linux” are associated with lower fraud risk.

The previous version of this model timed out during “GridSearchCV” — over 45 minutes without completing. The associated fix was narrowing the grid and confirming the saga solver had enough iterations. With those adjustments it ran in a reasonable amount of time.

Ensemble (Soft Voting & Stacking): Both ensemble methods combined all eight models from the team: the three above plus the Linear SVC and KNN from Model 4, and the Random Forest and Gradient Boosting from Model 1. The training set was resampled to balance the classes first (RandomUnderSampler, 8,823 samples per class). KNN and RBF SVM were trained on a 20% subsample of the balanced set because full training was too slow.

- **Soft Voting** averages the fraud probability from each model and picks the class with the higher average. No additional training is needed beyond the individual models.

- **Stacking** uses out-of-fold predictions from all eight base models as inputs to a Logistic Regression meta-learner. The meta-learner learns which base models to rely on more, which is why it slightly outperforms voting.

Model	Accuracy	Recall	Precision	F1	ROC-AUC
Decision Tree	0.7498	0.7294	0.0315	0.0604	0.8040
Naive Bayes	0.6976	0.8132	0.0290	0.0560	0.8239
Logistic Reg	0.8043	0.7865	0.0429	0.0814	0.8767
Linear SVC	0.8043	0.7874	0.0430	0.0815	0.8765
KNN	0.6938	0.7897	0.0279	0.0538	0.8064
RBF SVM	0.7881	0.8042	0.0406	0.0773	0.8730
Random Forest	0.8113	0.7838	0.0443	0.0839	0.8805
Grad Boosting	0.8197	0.8114	0.0478	0.0903	0.8953
Soft Voting	0.7948	0.8137	0.0423	0.0804	0.8846

Stacking (best)	0.8170	0.8178	0.0475	0.0897	0.8955

Table 9: Model Comparison (All Models)

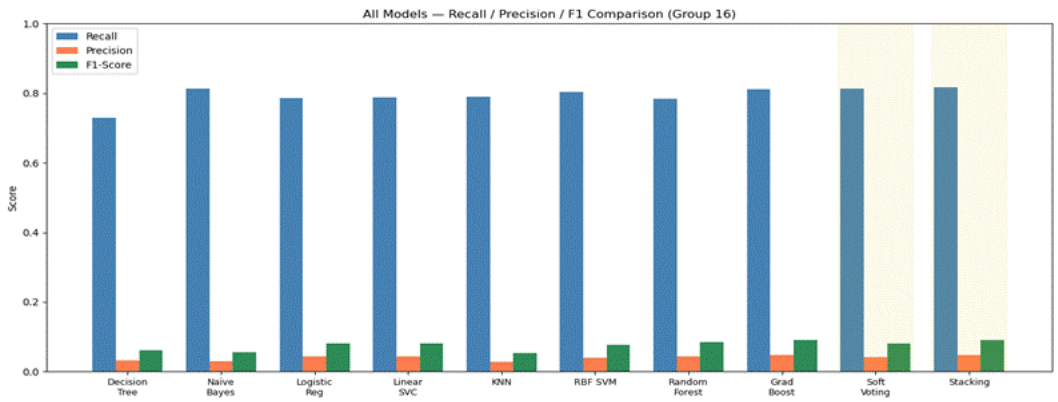

Figure 15: Recall / Precision / F1 across all models (ensemble columns highlighted)

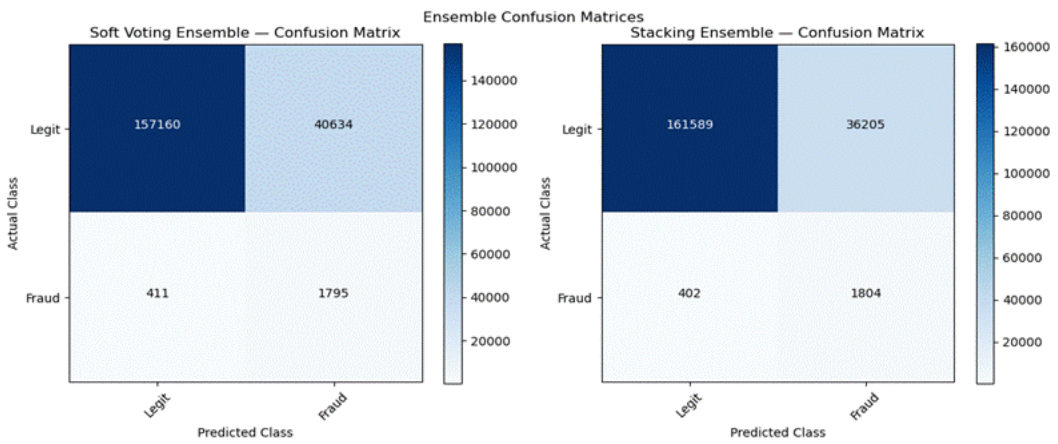

Figure 16: Soft Voting and Stacking confusion matrices

Model 3 Conclusion: The highest F1 of 0.0897 along with highest ROC-AUC of 0.8955, and highest recall of 81.78% are because of Stacking. Gradient Boosting and Logistic Regression outputs were weighed more heavily from the meta learner, as the two had strongest individual performance. Not too far behind is soft voting with recall of 81.37% and ROC-AUC of 0.8846. Both ensemble methods outperformed every individual model on every metric, and that is confirmation that merging the approaches is far better than relying on any of the ones as single method.

Model 4 (Sawyer McKenney): Fraud Detection using SVM, KNN, and Stacking

Model Selection: Linear SVM was chosen because after one-hot encoding, 51 dimensions were present. SVMs are designed to find decisions boundaries in high dimensional amounts of data. Since this project is dealing with a large amount of class imbalance, and because it penalizes misclassifying the minority more, SVM was assumed to be great for this purpose. As a result, SVM was able to achieve a high recall of 0.78.

KNN was chosen because it does not make assumptions based on the distributions of the data it focuses on localized irregularities that SVM might have missed. So as a result, it was assumed that if these methods were stacked, it would be possible to leverage the strengths of both SVM and KNN. When it was observed that stacking provided the highest F1 score, these assumptions were proven to be correct.

Model assumptions: For SVM, LinearSVC was used, assuming that fraud and not fraud can be separated by a hyperplane that has a 51-feature dimensional space. It is also assumed that missing a fraud case is more costly than a false alarm, so "class_weight" was set to be "balanced."

For KNN it was assumed that similar transactions are near each other. In other words, it was believed that the fraud transactions will have similar fraud neighbors. It was also assumed that all 51 features are equally informative.

Hyperparameter Tuning:

For SVM, "class_weight" was adjusted to be "balanced" so that the model did not predict "non-fraud" for everything. The max iterations was also adjusted to be 5000 instead of 1000 to give the optimizer enough iterations to converge on the large dataset.

For KNN, the only hyperparameter change that was made consisted of using all available CPU cores for faster computation by running in parallel.

Walkthrough:

During the preprocessing of this model, the "–1" value was replaced with "NaN" for the features with large amounts of missing data. "-1" and "NaN" were the values that represented missing values. Secondly, median imputation and normalization were applied in other features. One-hot encoding was also used on categorical features, and we resulted in 51 features after encoding.

The techniques applied in this model include Normalization and Bagging. Normalization was required for a distance-based model approach to apply to SVM and KNN.

Bootstrapping was used to try and reduce variances through aggregation, and it was applied to both SVM and KNN. For training, a 20/80 split was implemented, training by randomly selecting 80% of the data and then testing on the 20%.

As a result, it was found that SVM had achieved the highest recall with 77% but very low precision of about 4%. This means that the model was catching a large amount of fraud but also generating a lot of false alarms. On the other hand, KNN struggled with class imbalance and presented a 41% recall and 4% precision.

After stacking precision was able to be improved by 7% over the individual models and, was able to balance recall to 60%. Stacking also provided the best F1 score for this model at 0.13.

The observed takeaways from this model were that SVM and KNN are both distance-based models that struggle with high imbalance. With this in mind, the assumption is that tree-based methods like random forest would likely perform better for this problem but were outside the scope of this model.

Model	Recall	Precision	F-1 Score	Accuracy
Linear SVM	0.7774	0.0427	0.0810	0.81
KNN	0.0068	0.1744	0.0131	0.99
Stacking	0.6052	0.0703	0.1259	0.91

Table 10: Model Performance Comparison

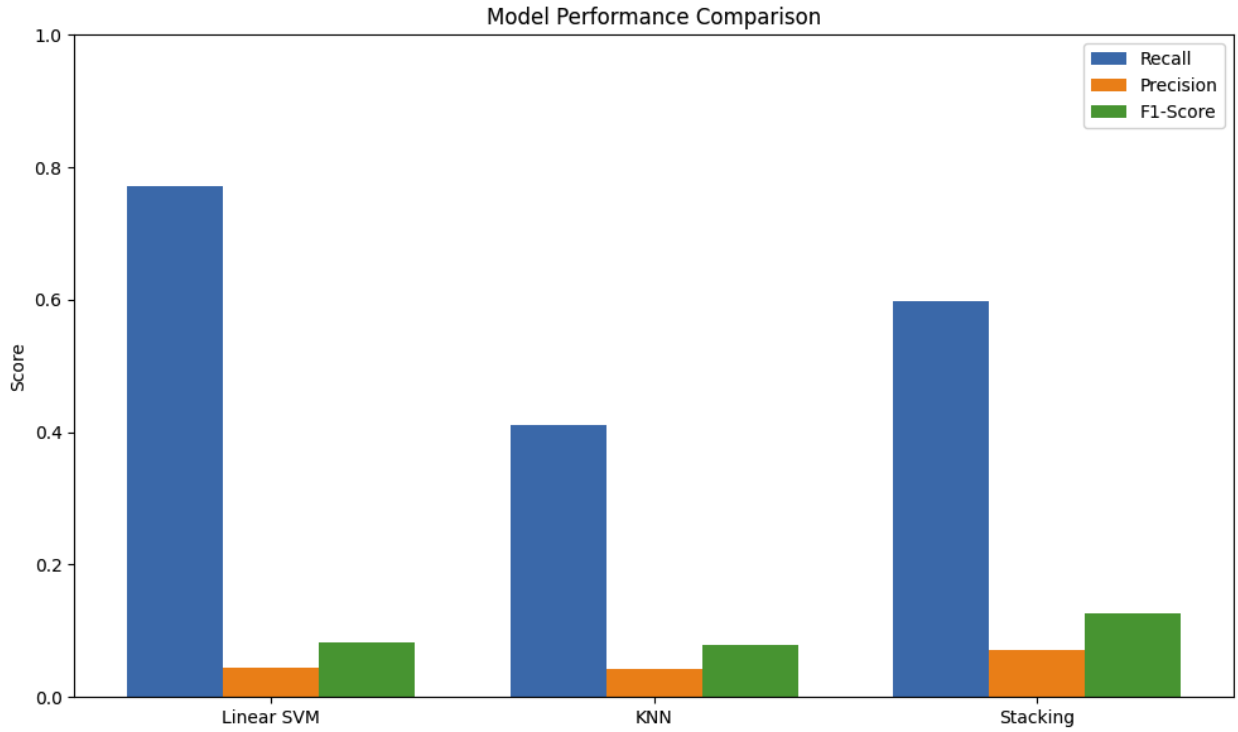

Figure 17: Model Performance Comparison

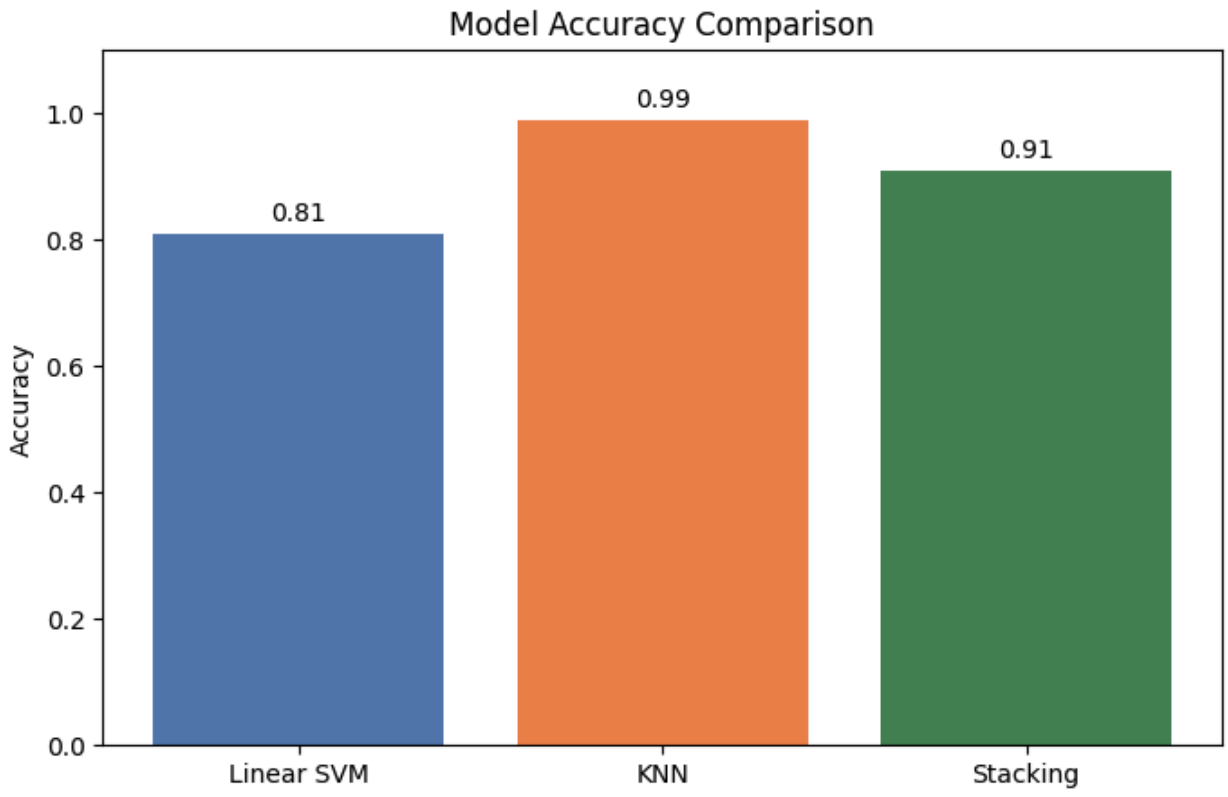

Figure 18: Model Accuracy Comparison

Linear SVM			KNN			Stacking		
	Not	**Fraud**		**Not**	**Fraud**		**Not**	**Fraud**
Not	160674	37120	**Not**	177639	20155	**Not**	180416	17378
Fraud	505	1701	**Fraud**	1298	908	**Fraud**	887	1319

Table 11: Model Confusion Matrices

Conclusion

Based on the four model implementations in this milestone. It is clear that the stacking model from all eight base models (Model 3) preformed the best. With a recall of 81.78% and a F-1 score of 0.0897. This goes to show that combining different types of learners through a meta-learning process produces a better fraud detection than any single algorithm. The best standalone model was gradient boosting. This achieved a F-1 score of 0.0903 and a ROC-AUC of 0.8955 telling how well the model separates the two classes. This was followed by Logistic Regression and Linear SVC that achieved 78% recall. Another thing worth noting is that in Model 1, the imbalanced test set Logistic regression provided the be the most reliable model on full imbalanced sets because it was able to achieve a F1-Score of 8% while still maintaining 80% recall. The other balanced training models achieved very high recall but as a result precisions dropped to 2%, thus driving the F1-scores down to 4-5%. Logistic regression was able to hold a better recall/precision balance on the imbalanced data set makis it the most practical model base.

KNN was the weakest predictive model in Model 4's implementation, failing significantly with class imbalance and producing poor recall and uncertain performance overall. The standalone Decision Tree was also a consistent underperformer compared to other models, especially in terms of maintaining a balance between precision and recall

From Model 2, additional insights were attained beyond pure predictions. The DT model shows that fraud detection is heavily influenced by a mix of behavioral features such as transaction velocity and session activity, along with financial attributes like credit risk score and proposed credit limit. The model achieved a high recall (~74%), meaning it was able to capture a large portion of fraud cases, but precision remained extremely low (~3%), resulting in a low F1-score (~0.06). This shows the impact of class imbalance, where the model prioritizes catching the fraud at the cost of a higher number of false positives. In comparison to Model 2's DT results, Model 3 showed a more balanced improvement after tuning, particularly in terms of F1-score and precision. Model 2 prioritized recalls (~74%) but suffered from extremely low precision (~3%), leading to a much lower F1-score (~.065), whereas Model 3 increased the recall to around 26% while improving precision, resulting in a higher F1-score (~.14). This contrast shows the two different modeling trade-offs. Model 3's approach aimed for a more balanced classifier whilst Model 2 focused more heavily on capturing as much fraud as possible, even if false positive rates were high. Despite the difference, both models show similar important features such as housing status and device type, reinforcing that fraud is driven by a mix of behavioral and identity-related factors.